

functions :

- **make_moons** : makes 2 interleaving circles, simple dataset to visualise clustering and classifications algorithms, returns the generated samples represented as an array of points X and labels y (each point from X has a label).
- **train_test_split** : splits the samples for training and testing sets, **important** : the stratify parameter ensures that both sets have the same proportions of classes (example if we have 900 pics of cats compared to 100 pics of dogs both the training and the testing sets have the same percentages of images of each class treated)
- **StratifiedKFold** : cross-validation, Random splitting techniques like **train_test_split()** or regular K-Fold can create problems if they produce imbalanced class proportions in the training and test sets.
- **StandardScaler** : normalizes the data by putting the mean to 0 and the variance to 1 <https://scikit-learn.org/stable/modules/generated/sklearn.preprocessing.StandardScaler.html#sklearn.preprocessing.StandardScaler>
we'll use a fit function to calculate the mean and the écart-type and the transform to change the data to normalised one. it's advised to use the fit on the train data (required even) then we transform both the training and testing data.
- **f.keras.Input** : initiate a keras tensor, which a symbolic tensor. it's treated as a placeholder, it holds no numerical data just shape and dtype and can be used for building a functional model (but not in computation) where we only know the inputs and outputs.
https://www.tensorflow.org/api_docs/python/tf/keras/Input
- **Dense** : creates a dense layer, uses and activation function check this video to understand the different activation function that'll help determine the best one to use based on our data <https://www.youtube.com/watch?v=s-V7gKrsels&t=85s> it's also on the cours.
in short though the video says that :
 - **Input** : most of the time though we often use Relu, it doesn't squeeze information. but in random cases it's boundary's doesn't change **Dying Relu Problem : blocks inputs less than 0** Elu and leaky ReLU do the job in this case.
 - **Output** : classification we use a softmax when we have 2 classes, in case the outputs are numbers above 0 like age/height,...ect we can use ReLU.

⚠ Notes :

- always **normalize** train and test sets !!!
- batch size affects learning dynamics, personal opinion 16/32 is good usually
- output from sigmoid is **probability**. forgetting to convert it before classification metrics is just blatantly incorrect.

more to be added later so far this is just the part for a **first neural network**

Encountered Difficulties : working locally on jupyter notebook with newer version of tensorflow/ numpy causes some timeout / hanging. new concepts and slow progress on the TP .

Total time spent on TP so far : ~5-6 hours